DAY 33 | POST-READ

LLMOps: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
LLM lifecycle	Scope, build, evaluate, deploy, operate, and loop back.
AgentOps (NEW v4)	Version prompts, tools, skills, MCP servers and models in one manifest with hashes.
Deprecation runbook (NEW v4)	Watch, inventory, evaluate, canary, switch, record.
Failure modes	Rate limits, timeouts, model updates, cost spikes, retrieval breaks, tool changes.
Resilience	Backoff with jitter, fallbacks, circuit breakers, timeouts everywhere.
MLOps vs LLMOps	LLMOps adds prompts, provider models, token costs, judges and drift.
Fine-tuning	For behaviour, format or efficiency at volume; not for fresh facts.
Cost and latency	Tokens x price; p50 and p95; trim, cache, route, cap agent steps.

Part B: Going beyond today

1. Pin model versions

Where the provider offers dated snapshots, use them in production and upgrade deliberately after running your golden set, instead of letting an alias change underneath you.

2. Idempotent retries

Retrying a request that triggers an action (send email, issue refund) can duplicate it. Use idempotency keys or check whether the action already happened before retrying.

3. Distillation and parameter-efficient fine-tuning

Use a large model to produce reviewed, high-quality outputs, then fine-tune a smaller model on them. For open models, LoRA trains small adapter layers instead of the whole model.

4. Skills and MCP servers are supply chain (NEW v4)

A skill or MCP server you did not write can change behaviour or leak data. Pin versions, review changes like code, and keep an allow-list. This links back to the Day 23 SME session.

5. Budgets and alerts

Set daily cost budgets per feature and alert at 80 percent. A runaway agent loop can burn a month's budget in hours.

Part C: Check yourself

- What belongs in an AgentOps release manifest, and why hash each part?
- List the six steps of a model deprecation runbook.
- Which errors should you retry, and which should you never retry?
- Give one case where fine-tuning is right and one where RAG is right.
- How would you trade latency against accuracy for a refund decision?

Part D: Practice ideas for tonight

- Write a release manifest for SmartDesk with every prompt, tool and model it uses.
- Write a deprecation runbook entry for the model your SmartDesk assistant uses today.
- Add a circuit breaker to your SmartDesk LLM client.

Coming up on Day 34: LLM deployment and lifecycle: managed agent runtimes, AI gateways, environments, secrets, streaming and canary releases.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today